EX NO: **SIMPLE NETWORK FUNCTIONS**

DATE:

AIM:

To understand and practise common network utility commands such as ping, netstat, ifconfig/ipconfig, nslookup, traceroute/tracert and getmac.

PROCEDURE:

1. PING:

The ping command checks whether a remote host can be reached and measures how long a packet takes to travel to it and back. It sends ICMP echo request packets to the destination and waits for the echo replies, which makes it one of the first tools used to troubleshoot a network problem.

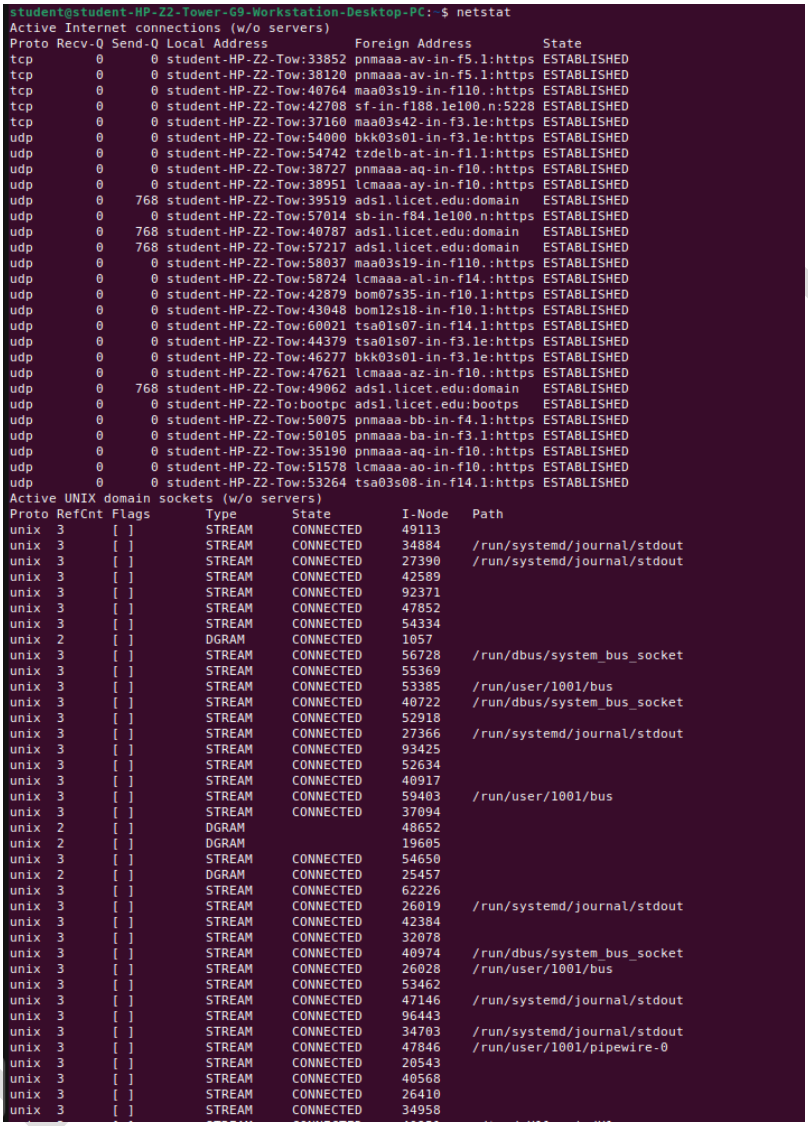
Syntax: ping <address>

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ ping 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.043 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.040 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.047 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.046 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.059 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.053 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.044 ms
64 bytes from 127.0.0.1: icmp_seq=11 ttl=64 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=12 ttl=64 time=0.053 ms
64 bytes from 127.0.0.1: icmp_seq=13 ttl=64 time=0.054 ms
64 bytes from 127.0.0.1: icmp_seq=14 ttl=64 time=0.042 ms
64 bytes from 127.0.0.1: icmp_seq=15 ttl=64 time=0.053 ms
64 bytes from 127.0.0.1: icmp_seq=16 ttl=64 time=0.036 ms
64 bytes from 127.0.0.1: icmp_seq=17 ttl=64 time=0.051 ms
64 bytes from 127.0.0.1: icmp_seq=18 ttl=64 time=0.052 ms
64 bytes from 127.0.0.1: icmp_seq=19 ttl=64 time=0.039 ms
64 bytes from 127.0.0.1: icmp_seq=20 ttl=64 time=0.040 ms
64 bytes from 127.0.0.1: icmp_seq=21 ttl=64 time=0.038 ms
64 bytes from 127.0.0.1: icmp_seq=22 ttl=64 time=0.053 ms
64 bytes from 127.0.0.1: icmp_seq=23 ttl=64 time=0.052 ms
64 bytes from 127.0.0.1: icmp_seq=24 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: icmp_seq=25 ttl=64 time=0.032 ms
64 bytes from 127.0.0.1: icmp_seq=26 ttl=64 time=0.052 ms
64 bytes from 127.0.0.1: icmp_seq=27 ttl=64 time=0.042 ms
^C
--- 127.0.0.1 ping statistics ---
27 packets transmitted, 27 received, 0% packet loss, time 26620ms
rtt min/avg/max/mdev = 0.015/0.044/0.059/0.009 ms
```

2. NETSTAT:

Netstat (network statistics) lists the active network connections of a system along with its routing tables and interface statistics. It is helpful for watching network activity and for tracking down connection-related faults.

Syntax: netstat [option]



```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 student-HP-Z2-Tow:33852 pnmaaa-av-in-f5.1:https ESTABLISHED
tcp        0      0 student-HP-Z2-Tow:38120 pnmaaa-av-in-f5.1:https ESTABLISHED
tcp        0      0 student-HP-Z2-Tow:40764 maa03s19-in-f110.:https ESTABLISHED
tcp        0      0 student-HP-Z2-Tow:42708 sf-in-f188.1e100.n:5228 ESTABLISHED
tcp        0      0 student-HP-Z2-Tow:37160 maa03s42-in-f3.1e:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:54000 bkk03s01-in-f3.1e:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:54742 tzdelb-at-in-f1.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:38727 pnmaaa-aq-in-f10.:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:38951 lcmaaa-ay-in-f10.:https ESTABLISHED
udp        0    768 student-HP-Z2-Tow:39519 ads1.licet.edu:domain   ESTABLISHED
udp        0      0 student-HP-Z2-Tow:57014 sb-in-f84.1e100.n:https ESTABLISHED
udp        0    768 student-HP-Z2-Tow:40787 ads1.licet.edu:domain   ESTABLISHED
udp        0    768 student-HP-Z2-Tow:57217 ads1.licet.edu:domain   ESTABLISHED
udp        0      0 student-HP-Z2-Tow:58037 maa03s19-in-f110.:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:58724 lcmaaa-al-in-f14.:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:42879 bom07s35-in-f10.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:43048 bom12s18-in-f10.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:60021 tsa01s07-in-f14.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:44379 tsa01s07-in-f3.1e:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:46277 bkk03s01-in-f3.1e:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:47621 lcmaaa-az-in-f10.:https ESTABLISHED
udp        0    768 student-HP-Z2-Tow:49062 ads1.licet.edu:domain   ESTABLISHED
udp        0      0 student-HP-Z2-To:bootpc ads1.licet.edu:bootps   ESTABLISHED
udp        0      0 student-HP-Z2-Tow:50075 pnmaaa-bb-in-f4.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:50105 pnmaaa-ba-in-f3.1:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:35190 pnmaaa-aq-in-f10.:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:51578 lcmaaa-ao-in-f10.:https ESTABLISHED
udp        0      0 student-HP-Z2-Tow:53264 tsa03s08-in-f14.1:https ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  3      [ ]         STREAM     CONNECTED     49113
unix  3      [ ]         STREAM     CONNECTED     34884    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     27390    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     42589
unix  3      [ ]         STREAM     CONNECTED     92371
unix  3      [ ]         STREAM     CONNECTED     47852
unix  3      [ ]         STREAM     CONNECTED     54334
unix  2      [ ]         DGRAM      CONNECTED     1057
unix  3      [ ]         STREAM     CONNECTED     56728    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     55369
unix  3      [ ]         STREAM     CONNECTED     53385    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     40722    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     52918
unix  3      [ ]         STREAM     CONNECTED     27366    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     93425
unix  3      [ ]         STREAM     CONNECTED     52634
unix  3      [ ]         STREAM     CONNECTED     40917
unix  3      [ ]         STREAM     CONNECTED     59403    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     37094
unix  2      [ ]         DGRAM                    48652
unix  2      [ ]         DGRAM                    19605
unix  3      [ ]         STREAM     CONNECTED     54650
unix  2      [ ]         DGRAM      CONNECTED     25457
unix  3      [ ]         STREAM     CONNECTED     62226
unix  3      [ ]         STREAM     CONNECTED     26019    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     42384
unix  3      [ ]         STREAM     CONNECTED     32078
unix  3      [ ]         STREAM     CONNECTED     40974    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     26028    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     53462
unix  3      [ ]         STREAM     CONNECTED     47146    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     96443
unix  3      [ ]         STREAM     CONNECTED     34703    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     47846    /run/user/1001/pipewire-0
unix  3      [ ]         STREAM     CONNECTED     20543
unix  3      [ ]         STREAM     CONNECTED     40568
unix  3      [ ]         STREAM     CONNECTED     26410
unix  3      [ ]         STREAM     CONNECTED     34958
```

3. IFCONFIG / IPCONFIG:

The ifconfig command (Linux/Unix) and the ipconfig command (Windows) show the configuration of the network interfaces, including IP addresses, subnet masks, broadcast addresses and MAC addresses.

Syntax: ifconfig (Linux) or ipconfig (Windows)

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ ifconfig
eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.227  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::73dd:c5e1:2a05:fd21  prefixlen 64  scopeid 0x20<link>
        ether 64:4e:d7:6e:6a:3e  txqueuelen 1000  (Ethernet)
        RX packets 97154  bytes 88073158 (88.0 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 37918  bytes 13095167 (13.0 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 19  memory 0x82280000-822a0000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 2948  bytes 290560 (290.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 2948  bytes 290560 (290.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wls5f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.22.241  netmask 255.255.252.0  broadcast 192.168.23.255
        inet6 fe80::9982:62ea:f379:2dd5  prefixlen 64  scopeid 0x20<link>
        ether d4:e9:8a:05:e7:bd  txqueuelen 1000  (Ethernet)
        RX packets 2015  bytes 209111 (209.1 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1079  bytes 126303 (126.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4. NSLOOKUP:

Nslookup is a command-line utility that sends queries to a DNS (Domain Name System) server. It converts a host name into its IP address and an IP address back into a host name, and is widely used to check DNS records and diagnose name-resolution problems.

Syntax: nslookup <domain name>

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ nslookup google.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.192.110
Name:   google.com
Address: 2404:6800:4002:819::200e
```

5. TRACEROUTE / TRACERT:

The traceroute command (Linux/Unix) and the tracert command (Windows) follow the path taken by packets from your computer to a destination host. They list every router (hop) along the way together with the time taken to reach each one.

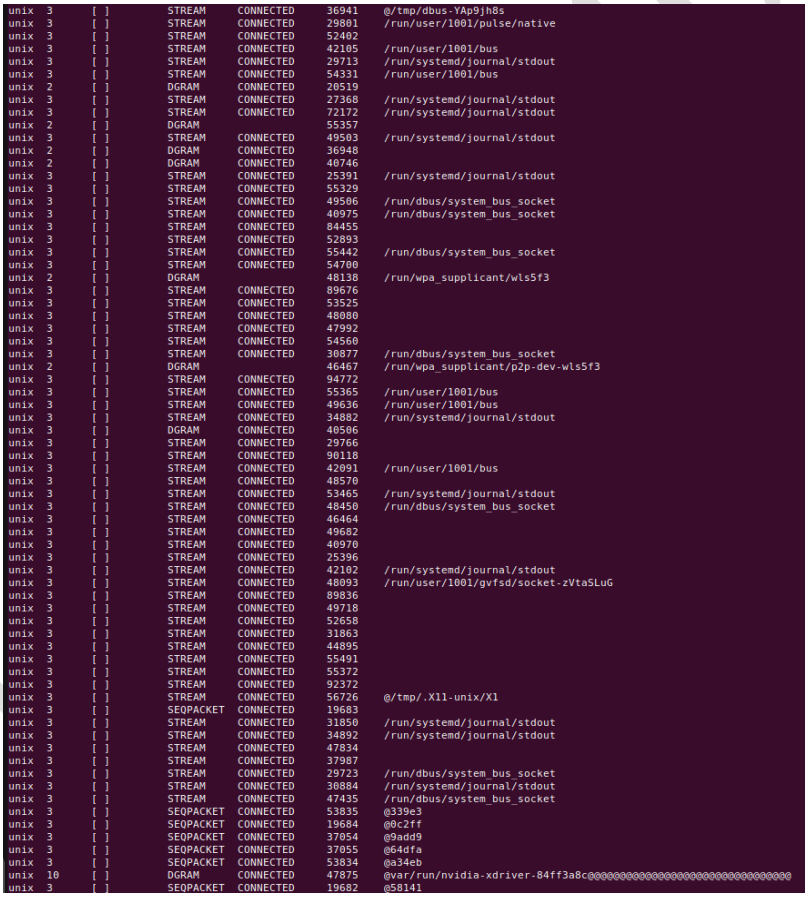
Syntax: traceroute <address> (Linux) or tracert <address> (Windows)

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ traceroute 127.0.0.1
traceroute to 127.0.0.1 (127.0.0.1), 64 hops max
  1   127.0.0.1  0.001ms  0.000ms  0.001ms
```

6. GETMAC:

Getmac is a Windows command that displays the MAC (Media Access Control) addresses of all the network adapters present in the computer. A MAC address is the unique hardware identifier given to every network interface.

Syntax: getmac



RESULT:

Hence, the basic network utility commands were executed successfully, and their respective outputs were observed and verified.

EX NO: **CYCLIC REDUNDANCY CHECK**

DATE : 29.7.2026

AIM:

To implement the Cyclic Redundancy Check (CRC) method and detect errors in transmitted data.

ALGORITHM:

1. Start the program.
2. Input the data bits and the generator polynomial.
3. Append (generator length - 1) zeros to the data.
4. Perform binary (modulo-2) XOR division on the padded data using the generator.
5. Use the remainder as the CRC and append it to the original data to form the code word.
6. At the receiver, input the received code word.
7. Divide the received code word using the same generator.
8. If the remainder contains only zeros, display "No Error"; otherwise display "Error Detected".
9. Stop the program.

PROGRAM:

```
import java.util.*;

public class CRC {

    // Binary (modulo-2) division
    static String xorDivide(String dividend, String divisor) {
        int len = divisor.length();
        char[] bits = dividend.toCharArray();

        for (int i = 0; i <= bits.length - len; i++) {
            if (bits[i] == '1') {
                for (int j = 0; j < len; j++) {
                    bits[i + j] = (bits[i + j] == divisor.charAt(j)) ? '0' : '1';
                }
            }
        }
        return new String(bits, bits.length - len + 1, len - 1);
    }

    // Sender: append CRC bits to the message
    static String encode(String message, String poly) {
        String padded = message + "0".repeat(poly.length() - 1);
```

3111241040 10

```
        String remainder = xorDivide(padded, poly);
        String frame = message + remainder;

        System.out.println("\n===== SENDER =====");
        System.out.println("Padded data   : " + padded);
        System.out.println("CRC remainder : " + remainder);
        System.out.println("Code word     : " + frame);
        return frame;
    }

    // Receiver: verify the incoming code word
    static void verify(String frame, String poly) {
        String remainder = xorDivide(frame, poly);
        boolean clean = remainder.chars().allMatch(c -> c == '0');

        System.out.println("\n===== RECEIVER =====");
        System.out.println("Remainder     : " + remainder);
        System.out.println(clean ? "Result       : No Error" : "Result       : Error Detected");
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.print("Enter data bits: ");
        String message = in.next();
        System.out.print("Enter generator polynomial bits: ");
        String poly = in.next();

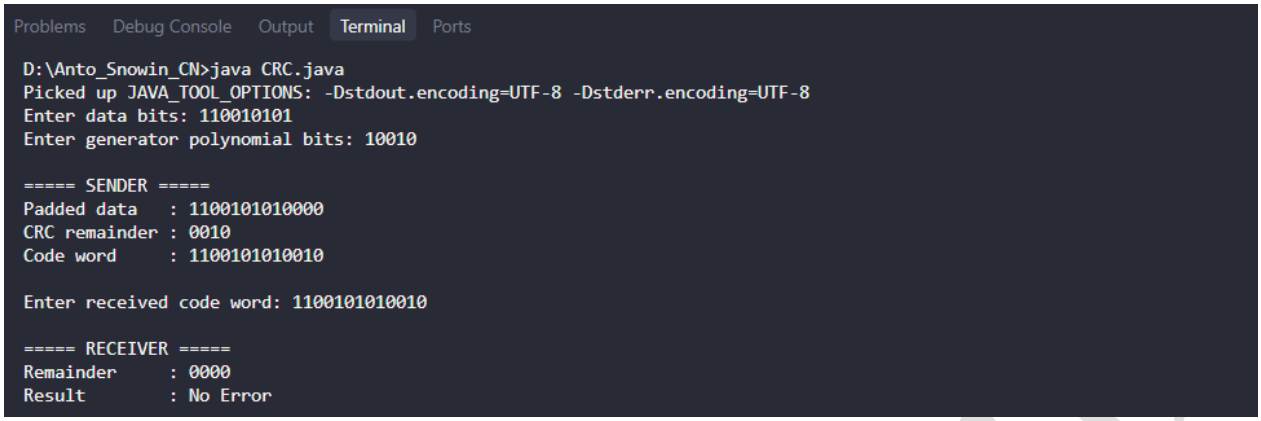
        encode(message, poly);

        System.out.print("\nEnter received code word: ");
        String received = in.next();
        verify(received, poly);

        in.close();
    }
}
```

3112410401O

OUTPUT:



RESULT:

Hence, the CRC technique was implemented successfully and the received data was checked for errors.

3111241040

EX NO: **SIMULATION OF ARP**

DATE:

AIM:

To implement the Address Resolution Protocol (ARP) and obtain the MAC address for a given IP address.

ALGORITHM:

1. Start the program.
2. Create an ARP table containing IP-to-MAC address mappings.
3. Get the required IP address as input.
4. Search for the entered IP address in the ARP table.
5. If the entry is available, retrieve and display its MAC address.
6. If no entry is available, display a suitable "not found" message.
7. Stop the program.

PROGRAM:

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class ARP {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        // ARP table : IP address -> MAC address
        Map<String, String> arpTable = new HashMap<>();
        arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
        arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
        arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");

        System.out.print("Enter the IP address: ");
        String ipAddr = in.nextLine().trim();

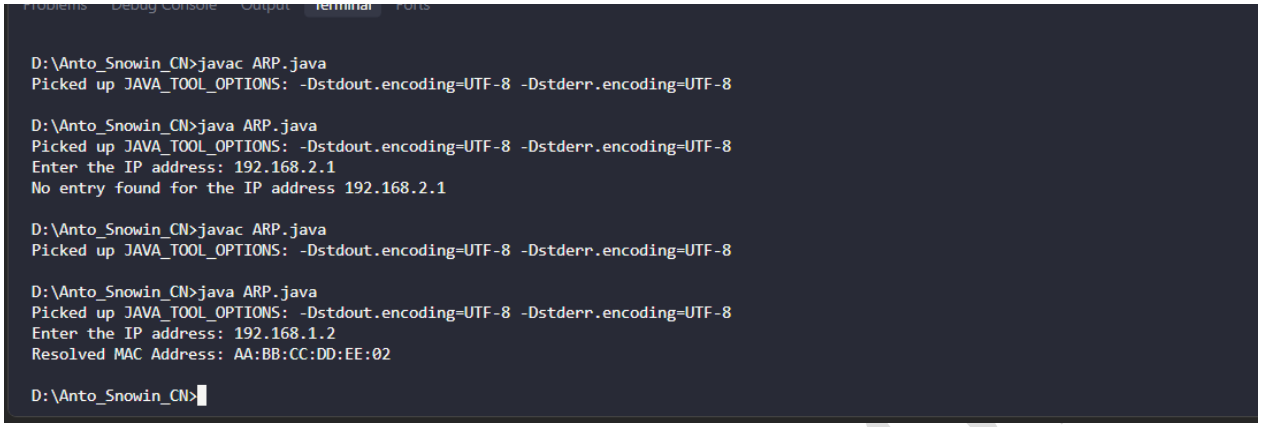
        if (arpTable.containsKey(ipAddr)) {
            System.out.println("Resolved MAC Address: " + arpTable.get(ipAddr));
        } else {
            System.out.println("No entry found for the IP address " + ipAddr);
        }

        in.close();
    }
```

3112410401O

}

OUTPUT:



RESULT:

Hence, the ARP protocol was simulated successfully and the corresponding MAC address was obtained.

3111241040

EX NO: **SIMULATION OF RARP**

DATE:

AIM:

To implement the Reverse Address Resolution Protocol (RARP) and find the IP address associated with a given MAC address.

ALGORITHM:

1. Start the program.
2. Create a RARP table containing MAC-to-IP address mappings.
3. Get the required MAC address as input.
4. Search for the entered MAC address in the RARP table.
5. If the entry is available, retrieve and display its IP address.
6. If no entry is available, display a suitable "not found" message.
7. Stop the program.

PROGRAM:

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class RARP {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        // RARP table : MAC address -> IP address
        Map<String, String> rarpTable = new HashMap<>();
        rarpTable.put("AA:BB:CC:DD:EE:01", "192.168.1.1");
        rarpTable.put("AA:BB:CC:DD:EE:02", "192.168.1.2");
        rarpTable.put("AA:BB:CC:DD:EE:03", "192.168.1.3");

        System.out.print("Enter the MAC address: ");
        String macAddr = in.nextLine().trim().toUpperCase();

        if (rarpTable.containsKey(macAddr)) {
            System.out.println("Resolved IP Address: " + rarpTable.get(macAddr));
        } else {
            System.out.println("No entry found for the MAC address " + macAddr);
        }

        in.close();
    }
```

3112410401O

}

OUTPUT:

```
D:\Anto_Snowin_CN>javac RARP.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

D:\Anto_Snowin_CN>java RARP.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the MAC address: AA:BB:CC:DD:EE:01
Resolved IP Address: 192.168.1.1

D:\Anto_Snowin_CN>javac RARP.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

D:\Anto_Snowin_CN>java RARP.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter the MAC address: AA:BB:CC:DD:EE:04
No entry found for the MAC address AA:BB:CC:DD:EE:04

D:\Anto_Snowin_CN>
```

3111241040

RESULT:

Hence, the RARP protocol was simulated successfully and the corresponding IP address was obtained.

EX NO: SIMULATION OF DISTANCE VECTOR ROUTING

DATE:

AIM:

To implement the Distance Vector Routing algorithm and determine the minimum-cost routes between routers.

ALGORITHM:

1. Start the program.
2. Input the number of routers.
3. Input the network cost matrix, using 999 when there is no direct link.
4. Initialize each router's distance table using the direct link costs.
5. Each router compares its current distance to every destination with the distance obtained through each of its neighbours.
6. If a lower-cost route is found, update the cost and next hop.
7. Repeat the comparison and update process until no table changes occur.
8. Display the final distance vector table for all routers.
9. Stop the program.

PROGRAM:

```
import java.util.Scanner;

public class DistanceVector {
    static final int INF = 999;

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.print("Enter number of routers: ");
        int n = in.nextInt();

        int[][] cost = new int[n][n];
        int[][] dist = new int[n][n];
        int[][] hop  = new int[n][n];

        System.out.println("Enter cost matrix (use 999 for no direct link):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cost[i][j] = in.nextInt();
                dist[i][j] = cost[i][j];
                hop[i][j] = (cost[i][j] < INF) ? j : -1;
```

3111241040 10

```
            }
        }

        // Every router repeatedly learns from its neighbours
        boolean changed;
        do {
            changed = false;
            for (int i = 0; i < n; i++) {
                for (int nb = 0; nb < n; nb++) {
                    if (i == nb || cost[i][nb] >= INF) continue;
                    for (int j = 0; j < n; j++) {
                        if (dist[nb][j] >= INF) continue;
                        int viaNb = cost[i][nb] + dist[nb][j];
                        if (viaNb < dist[i][j]) {
                            dist[i][j] = viaNb;
                            hop[i][j] = nb;
                            changed = true;
                        }
                    }
                }
            }
        } while (changed);

        System.out.println("\nFinal Distance Vector Tables");
        for (int i = 0; i < n; i++) {
            System.out.println("\nRouter " + i);
            System.out.println("Dest\tCost\tNext Hop");
            for (int j = 0; j < n; j++) {
                String via = (hop[i][j] == -1) ? "-" : String.valueOf(hop[i][j]);
                String c = (dist[i][j] >= INF) ? "INF" : String.valueOf(dist[i][j]);
                System.out.println(j + "\t" + c + "\t" + via);
            }
        }
        in.close();
    }
}
```

3112410401O

OUTPUT:

```
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter number of routers: 4
Enter cost matrix (use 999 for no direct link):
0 2 99 1
2 0 3 8
4 83 0 1
82 34 1 0

Final Distance Vector Tables

Router 0
Dest    Cost    Next Hop
0       0       0
1       2       1
2       2       3
3       1       3

Router 1
Dest    Cost    Next Hop
0       2       0
1       0       1
2       3       2
3       3       0

Router 2
Dest    Cost    Next Hop
0       4       0
1       6       0
2       0       2
3       1       3

Router 3
Dest    Cost    Next Hop
0       5       2
1       7       2
2       1       2
3       0       3
```

3112410

RESULT:

Hence, the Distance Vector Routing algorithm was implemented successfully and the minimum distances between routers were determined.

EX NO: SIMULATION OF LINK STATE ROUTING

DATE:

AIM:

To implement Link State Routing using Dijkstra's algorithm to determine the shortest paths in the network.

ALGORITHM:

1. Start the program.
2. Input the number of routers.
3. Input the network cost matrix, where 0 represents no link.
4. Input the source router.
5. Set the source distance to 0 and initialize all other distances to infinity.
6. Choose the unvisited router with the smallest distance and mark it as visited.
7. Update neighbouring router distances whenever a shorter path is available.
8. Continue until all reachable routers have been visited.
9. Display the cost and path from the source to each router.
10. Stop the program.

PROGRAM:

```
import java.util.Arrays;
import java.util.Scanner;

public class LinkState {
    static final int INF = 9999;

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.print("Enter number of routers: ");
        int n = in.nextInt();

        int[][] graph = new int[n][n];
        System.out.println("Enter cost matrix (0 = no link):");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                graph[i][j] = in.nextInt();

        System.out.print("Enter source router (0 to " + (n - 1) + "): ");
        int src = in.nextInt();
        if (src < 0 || src >= n) {
```

3112410410

```
            System.out.println("Invalid source router.");
            return;
        }

        int[] dist = new int[n];
        int[] prev = new int[n];
        boolean[] done = new boolean[n];
        Arrays.fill(dist, INF);
        Arrays.fill(prev, -1);
        dist[src] = 0;

        for (int round = 0; round < n; round++) {
            // pick the closest router not yet finalised
            int u = -1;
            for (int i = 0; i < n; i++)
                if (!done[i] && (u == -1 || dist[i] < dist[u]))
                    u = i;
            if (u == -1 || dist[u] == INF) break;
            done[u] = true;

            // relax the links of u
            for (int v = 0; v < n; v++) {
                if (graph[u][v] != 0 && dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                    prev[v] = u;
                }
            }
        }

        System.out.println("\nShortest paths from router " + src + ":");
        for (int i = 0; i < n; i++) {
            if (dist[i] == INF) {
                System.out.println(src + " -> " + i + " : unreachable");
                continue;
            }
            StringBuilder path = new StringBuilder();
            for (int x = i; x != -1; x = prev[x])
                path.insert(0, x + (x == i ? "" : " -> "));
            System.out.println(src + " -> " + i + " : cost = " + dist[i] + "   path: " + path);
        }
        in.close();
    }
}
```

3112410401O

OUTPUT:

```
D:\Anto_Snowin_CN>javac LinkState.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

D:\Anto_Snowin_CN>java LinkState.java
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter number of routers: 4
Enter cost matrix (0 = no link):
0 2 0 1
2 0 3 0
0 3 0 5
1 0 5 0
Enter source router (0 to 3): 0

Shortest paths from router 0:
0 -> 0 : cost = 0   path: 0
0 -> 1 : cost = 2   path: 0 -> 1
0 -> 2 : cost = 5   path: 0 -> 1 -> 2
0 -> 3 : cost = 1   path: 0 -> 3
```

3112241040

RESULT:

Hence, Link State Routing was implemented successfully using Dijkstra's algorithm, and the shortest paths from the selected source router were obtained.

3111241040 10